

# Методичка: сценарий server-health.sh для диагностики Linux-сервера

**Темы:** Bash, functions, arguments, exit codes, set -euo pipefail, trap, grep, awk, logging, идемпотентность

**ОС сервера:** Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

**ОС клиента:** Debian/Ubuntu или Windows 10/11

**Среда:** VirtualBox

**Формат:** самостоятельная практическая работа

**Ориентировочное время:** 4 академических часа

## 1. Что настроим

В этой работе настраиваем часть Linux-инфраструктуры организации.

Команды выполняем на Debian/Ubuntu, а результат проверяем локально и с клиента.

Используем узлы: linux01.

Главная идея работы:

Bash-автоматизация считается готовой не тогда, когда скрипт выводит текст, а тогда, когда он безопасно повторяется, возвращает понятный exit code, пишет журнал и помогает быстро отличить OK, WARNING и CRITICAL.

## 2. Схема учебного стенда

| Узел     | Роль                  | ОС                        | Сетевые адаптеры      | IP-адрес         |
|----------|-----------------------|---------------------------|-----------------------|------------------|
| linux01  | основной сервер       | Debian/Ubuntu Server      | NAT + внутренняя сеть | 192.168.10.10/24 |
| client01 | клиент проверки       | Debian/Ubuntu или Windows | внутренняя сеть       | 192.168.10.20/24 |
| srv02    | дополнительный сервер | Debian/Ubuntu Server      | NAT + внутренняя сеть | 192.168.10.30/24 |

```
VirtualBox NAT
|
linux01 ---- внутренняя сеть linux-lab ---- client01/srv02
```

NAT нужен для установки пакетов. Учебные сервисы проверяем во внутренней сети. Если используем имена, заранее добавляем DNS или `/etc/hosts` и проверяем `getent hosts`.

### 3. Необходимые ресурсы и предварительные условия

Нужны:

- Debian/Ubuntu Server;

- права `sudo`;
- каталог `/usr/local/sbin`;
- службы для проверки, например `nginx` или `ssh`;
- журнал `/var/log/server-health.log`.

## 4. Подготовка виртуальных машин

```
hostnamectl  
ip -br link  
ip -br address  
ip route  
resolvectl status
```

Добавляем имена в `/etc/hosts`, если DNS ещё нет:

```
sudo nano /etc/hosts  
  
192.168.10.10 linux01.company.test linux01  
192.168.10.20 client01.company.test client01  
192.168.10.30 srv02.company.test srv02
```

Проверяем:

```
getent hosts linux01.company.test  
ping -c 3 client01.company.test
```

## Самопроверка этапа

- ВМ включены;
- NAT доступен для установки пакетов;
- внутренняя сеть работает;
- имена, используемые в командах, разрешаются.

*Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.*

## 5. Адресный план или план ресурсов

| Ресурс                | Значение                |
|-----------------------|-------------------------|
| Основной сервер       | linux01, 192.168.10.10  |
| Клиент                | client01, 192.168.10.20 |
| Дополнительный сервер | srv02, 192.168.10.30    |
| Учебный домен         | company.test            |
| Рабочие каталоги      | по сценарию             |

План работы:

- создать `/usr/local/sbin/server-health.sh`;
- разделить проверки на функции;
- добавить `-h`, аргументы и exit codes 0-3;
- проверить диски, inode, RAM, load, службы, порты, gateway, DNS и backup;
- добавить журналирование;
- проверить повторный безопасный запуск.

## 6. Исходная проверка

```
cat /etc/os-release
hostnamectl
```

```
ip -br address
ip route
systemctl --failed
ss -lntup
```

## Самопроверка этапа

- ОС определена;
- сеть работает;
- нет неожиданных failed-служб;
- исходное состояние записано.

## 7. Пошаговая настройка

### 7.1. Подготовка пакетов

```
sudo apt update
```

Если обновление индекса не работает, проверяем NAT, DNS и маршрут до репозиториев.

### 7.2. Основная настройка

Открываем сценарий:

```
sudo nano /usr/local/sbin/server-health.sh
```

Полный учебный текст:

```
#!/usr/bin/env bash
set -euo pipefail

LOG_FILE="/var/log/server-health.log"
STATUS=0

usage() {
    echo "Usage: server-health.sh [-h]"
}

log_msg() {
```

```

local level="$1"
local message="$2"
printf '%s [%s] %s\n' "$(date '+%F %T')" "$level" "$message" | tee -a
"$LOG_FILE"
}

check_disk() {
local used
used=$(df -P / | awk 'NR==2 {gsub("%", "", $5); print $5}')
if [ "$used" -ge 90 ]; then
log_msg CRITICAL "root filesystem usage is ${used}%"
STATUS=2
elif [ "$used" -ge 80 ]; then
log_msg WARNING "root filesystem usage is ${used}%"
[ "$STATUS" -lt 1 ] && STATUS=1
else
log_msg OK "root filesystem usage is ${used}%"
fi
}

check_service() {
local service="$1"
if systemctl is-active --quiet "$service"; then
log_msg OK "service $service is active"
else
log_msg CRITICAL "service $service is not active"
STATUS=2
fi
}

check_dns() {
if getent hosts deb.debian.org >/dev/null; then
log_msg OK "DNS lookup works"
else
log_msg CRITICAL "DNS lookup failed"
STATUS=2
fi
}

while getopts ":h" opt; do
case "$opt" in
h) usage; exit 3 ;;
*) usage; exit 3 ;;
esac
done

check_disk
check_service ssh
check_dns
exit "$STATUS"

```

Назначаем права и запускаем:

```
sudo chmod 750 /usr/local/sbin/server-health.sh
```

```
sudo /usr/local/sbin/server-health.sh
echo $?
sudo tail -n 20 /var/log/server-health.log
```

## Самопроверка этапа

```
sudo bash -n /usr/local/sbin/server-health.sh
sudo /usr/local/sbin/server-health.sh
echo $?
sudo tail -n 20 /var/log/server-health.log
```

Ожидаем exit code 0, 1, 2 или 3, а в журнале строки

OK/WARNING/CRITICAL.

Если результат отличается от ожидаемого, дальше пока не продолжаем.

Сначала проверяем этот этап.

## 7.3. Восстановление или откат

Перед изменением важных файлов создаём резервную копию. Для созданных сценариев и конфигов проверяем ручной запуск, код завершения и журнал. Если сценарий связан с безопасностью или backup, обязательно показываем восстановление.

## 8. Проверка итогового результата

```
sudo bash -n /usr/local/sbin/server-health.sh
sudo /usr/local/sbin/server-health.sh
echo $?
sudo tail -n 20 /var/log/server-health.log
```

Ожидаем exit code 0, 1, 2 или 3, а в журнале строки

OK/WARNING/CRITICAL.

Границы результата:

- работа выполняется в учебной сети;

- тестовые ключи, пароли и сертификаты нельзя переносить в промышленную среду;
- успешный запуск команды не заменяет проверку результата и журналов.

## 9. Диагностика типовых ошибок

| Симптом                               | Вероятная причина                            | Проверка                                      | Исправление                                         |
|---------------------------------------|----------------------------------------------|-----------------------------------------------|-----------------------------------------------------|
| Скрипт падает без понятного сообщения | Не обработана ошибка при <code>set -e</code> | <code>bash -x script</code>                   | Добавить проверку команды или условие               |
| Exit code всегда 0                    | STATUS не меняется при ошибке                | <code>echo \$?</code>                         | Исправить присваивание STATUS                       |
| Журнал не пишется                     | Нет прав на <code>/var/log</code>            | <code>ls -l /var/log/server-health.log</code> | Запускать через <code>sudo</code> или изменить путь |

|                          |                                          |                                                              |                             |
|--------------------------|------------------------------------------|--------------------------------------------------------------|-----------------------------|
| Пакет не устанавливается | Нет доступа к репозиториям               | <code>ip route, getent hosts deb.debian.org</code>           | Проверить NAT и DNS         |
| Служба не запускается    | Ошибка конфигурации                      | <code>systemctl status, journalctl -u &lt;service&gt;</code> | Исправить конфигурацию      |
| Клиент не подключается   | Порт не слушается или firewall блокирует | <code>ss -lntup, nc -vz &lt;ip&gt; &lt;port&gt;</code>       | Проверить службу и firewall |

## 10. Итоговая самостоятельная проверка

- ☐ Стенд соответствует схеме.
- ☐ Имена и адреса согласованы.
- ☐ Нужные пакеты установлены.
- ☐ Основная настройка выполнена.
- ☐ Проверочные команды подтверждают результат.
- ☐ Клиентская проверка проходит, если сервис сетевой.
- ☐ В журналах нет необъяснённых ошибок.
- ☐ Одна типовая ошибка найдена и исправлена.

## 11. Что приложить к отчёту

К отчёту прикладываем схему, адресный план, ключевой фрагмент конфигурации или сценария, вывод основной проверки, клиентский результат, журнал/диагностику и описание исправленной ошибки.

## **12. Контрольные вопросы**

1. Какую задачу решает технология этой лекции?
2. Какие исходные параметры нужно проверить до настройки?
3. Какая команда подтверждает основной результат?
4. Как отличить ошибку конфигурации от сетевой проблемы?
5. Почему нельзя хранить секреты в открытом виде?
6. Что проверить в журналах?
7. Какие ограничения учебного стенда важны?
8. Что включить в отчёт?